Python Programming Lab. [Batch-B3]

Assignment Submission Details

Field	Details
Student Name *	Dilip Balu Magar
Student PRN *	125M1H055
Course Name	Python Programming Lab. (MCA31PC06)
Academic Year	2025-26 (Semester-1)
Course Teacher	Prof. Prakash Ukhalkar
Assignment Name	Assignments based on Object Oriented Programming 1.
Assignment Number	Assignment 12
Submission Date *	11-12-2025

1.Create a Vehicle class with max_speed and mileage instance attributes and print max_speed and mileage.

In [ ]:
```
# Question 1:


class Vehicle:
    def __init__(self, max_speed, mileage):
      # def __init__(self): This method is automatically executed when you create a
        self.max_speed = max_speed     # we can also create instance variables outs
        self.mileage = mileage

# creating an object
car = Vehicle(180, 15000)     # internally converts this into => Vehicle.__init__(c

print("Max Speed:", car.max_speed)
print("Mileage:", car.mileage)


# __init__        =>Python’s constructor         Runs automatically when object is

#  self           =>Refers to the object        Same as Java’s this      Needed to
#self MUST be there because every instance method receives the object as first argu

# Python NEVER declares instance variables outside the constructor
#Python does not automatically differentiate instance vs local variables so wealway
```

```
Max Speed: 180
Mileage: 15000
```

In [4]:
```
#creating instance variable outside the class
class Vehicle:
    pass

v = Vehicle()

v.speed = 180      # creating instance variable outside the class
v.mileage = 15000

print("Max Speed:", v.speed)
print("Mileage:", v.mileage)
```

```
Max Speed: 180
Mileage: 15000
```

In [ ]:
```
#creating instace variables inside another method rather than __init__
class Vehicle:
    def set_details(self, speed, mileage):
        self.speed = speed
        self.mileage = mileage

v = Vehicle()  #call default constructor __init__()
v.set_details(150, 12000)  # internally converted into => Vehicle.set_details(v, 15


print("Max Speed:", v.speed)
print("Mileage:", v.mileage)
```

```
Max Speed: 150
Mileage: 12000
```

2.Write a Python class to find a pair of elements (indices of the two numbers) from a given array whose sum equals a specific target number.

In [ ]:
```
# Question 2:

class TwoSum:
    def __init__(self, nums):
        self.nums = nums

    def find_indices(self, target):
        hash_map = {}   # Stores: number → index

        for index, num in enumerate(self.nums):
          # enumerate() => It gives both index and value
            secondNumber = target - num   # find needed number to reach target

            if secondNumber in hash_map:
                return (hash_map[secondNumber], index)  # return indices

            hash_map[num] = index   # store current number

        return None  # no pair found
```

```
nums = [2, 7, 11, 15]
print(nums)
target = 9

obj = TwoSum(nums)
result = obj.find_indices(target)

if result is None:
    print(f"There are no numbers whose sum = {target}")
else:
    print(f"Numbers at indices {result} give sum = {target}")
```

```
[2, 7, 11, 15]
Numbers at indices (0, 1) give sum = 9
```

[3, 7, 11, 15] There are no numbers whose sum = 9

3.Create a Bus child class that inherits from the Vehicle class. The default fare charge of any vehicle is seating capacity * 100. If Vehicle is a Bus instance, we need to add an extra 10% on the full fare as a maintenance charge. So total fare for bus instance will become the final amount = total fare + 10% of the total fare.

In [ ]:
```
# Question 3:
class Vehicle:
    def __init__(self, capacity):
        self.capacity = capacity

    def fare(self):
        return self.capacity * 100


class Bus(Vehicle):
    def fare(self):
        total_fare = super().fare()      # get fare from Vehicle class
        maintenance_charge = total_fare * 0.10
        return total_fare + maintenance_charge


bus = Bus(50)
print("Bus capacity : ",bus.capacity)
print("Total Bus Fare:", bus.fare())
```

```
Bus capacity 50
Total Bus Fare: 5500.0
```

4.Write a Python class to convert an integer to a roman numeral.

In [ ]:
```
# Question 4:
class IntegerToRoman:
    def int_to_roman(self, num):

        values =    [1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1]
        symbols = ["M", "CM","D","CD","C","XC","L","XL","X","IX","V","IV","I"]
```

```
        roman = ""

        # Convert number to roman numeral
        for i in range(len(values)):
            while num >= values[i]:
                roman += symbols[i]
                num -= values[i]

        return roman



obj = IntegerToRoman()

print(f"Conversion of 2025 to Roman => {obj.int_to_roman(2025)}")   # MMXXIV
print(f"Conversion of 58 to Roman => {obj.int_to_roman(58)}")     # LVIII
print(f"Conversion of 10 to Roman => {obj.int_to_roman(10)}")   # MCMXCIV
```

```
Conversion of 2025 to Roman => MMXXV
Conversion of 58 to Roman => LVIII
Conversion of 10 to Roman => X
```

5.Write a Python class to find validity of a string of parentheses, '(', ')', '{', '}', '[' and ']. These brackets must be close in the correct order, for example "()" and "()[]{}" are valid but "[)", "({[)]" and "{{{" are invalid.

In [ ]:

```
# Question 5:
class Validator:
    def is_valid(self, s):
        stack = []

        # mapping of closing -> opening brackets
        match = {')': '(', '}': '{', ']': '['}

        for char in s:
            # If it is  opening bracket, push to stack
            if char in "({[":
                stack.append(char)

            # If it is closing bracket
            elif char in ")}]":
                # If stack is empty OR top does not match -> invalid
                if not stack or stack[-1] != match[char]:
                    return False
                stack.pop()  # remove the matched opening bracket

        # If stack is empty -> all brackets matched
        return len(stack) == 0


obj = Validator()

tests = ["()", "()[]{}", "[)", "((((", ")))))"]
```

```
for t in tests:
    print(f"{t} => {obj.is_valid(t)}")
```

```
() => True
()[]{} => True
[) => False
(((( => False
))))) => False
```

6.Write a Python program that defines a Car class. Include attributes like make, model, and year. Create an object of the class and print its details.

In [34]:
```
# Question 6:
class Car:
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year

    def display(self):
        print("Car Details")
        print(f"Make: {self.make}")
        print(f"Model: {self.model}")
        print(f"Year: {self.year}")


my_car = Car("Rolls Royce", "Phantom", 2023)


my_car.display()
```

```
Car Details
Make: Rolls Royce
Model: Phantom
Year: 2023
```

7.Create a program that defines a Student class. Include attributes for name, age, and grade. Write a method to display the student's information.

In [36]:
```
# Question 7:
class Student:
    def __init__(self, name, age, grade):
        self.name = name
        self.age = age
        self.grade = grade

    def display(self):
        print("Student Information:")
        print(f"Name  : {self.name}")
        print(f"Age   : {self.age}")
        print(f"Grade : {self.grade}")


stud = Student("Dilip", 22, "A")
```

```
stud.display()
```

```
Student Information:
Name  : Dilip
Age   : 22
Grade : A
```

8.Write a Python program that uses a constructor to initialize an object of a Person class with a given name and age. Print the details of the person.

In [38]:
```
# Question 8:
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print("Person Details:")
        print(f"Name : {self.name}")
        print(f"Age  : {self.age}")


p = Person("Dilip", 22)

p.display()
```

```
Person Details:
Name : Dilip
Age  : 22
```

9. Create a program that defines a Book class with a constructor that sets the book title and author. Use a destructor to print a message when an object is destroyed.

In [48]:
```
# Question 9:

class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author
        print(f"Book Created: {self.title} by {self.author}")

#In Python, destructors are not used very often because Python has automatic garbag
    def __del__(self):
        print(f"Book Destroyed: {self.title} by {self.author}")


b1 = Book("Java", "E. Balagurusamy")

print(f"Title : {b1.title}")
print(f"Author: {b1.author}")

#Garbage Collection does not always destroy objects at the exact moment the program

del b1       #force the destructor to run immediately
```

```
#b1=None

# del b1        Deletes the variable name b1
# b1 = None     Removes reference by assigning None
```

```
Book Created: Java by E. Balagurusamy
Title : Java
Author: E. Balagurusamy
Book Destroyed: Java by E. Balagurusamy
```

In [49]:

```
# Question 9:

class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author
        print(f"Book Created: {self.title} by {self.author}")

#In Python, destructors are not used very often because Python has automatic garbag
    def __del__(self):
        print(f"Book Destroyed: {self.title} by {self.author}")


b1 = Book("Java", "E. Balagurusamy")

print(f"Title : {b1.title}")
print(f"Author: {b1.author}")

#Garbage Collection does not always destroy objects at the exact moment the program

#del b1       #force the destructor to run immediately

b1=None

# b1=None => does NOT delete the variable deletes the object that b1 was pointing t
#  Destructor runs because the object has no references left
```

```
Book Created: Java by E. Balagurusamy
Title : Java
Author: E. Balagurusamy
Book Destroyed: Java by E. Balagurusamy
```

10. Write a Python program that demonstrates inheritance. Define a base class Animal with a method make_sound(). Create derived classes Dog and Cat that override the method.

In [56]:

```
# Python supports 5 types of inheritance:

# Single     # Multilevel  # Hierarchical    # Hybrid    # Multiple

#python support multiple inheritance unlike java because Python uses
# Method Resolution Order (MRO) and the C3 linearization algorithm to resolve confl

# Question 10:

class Animal:
```

```python
    def make_sound(self):
        print("Animal makes a sound")

class Dog(Animal):
    def make_sound(self):
        print("Dog barks")


class Cat(Animal):
    def make_sound(self):
        print("Cat meows")


a = Animal()
d = Dog()
c = Cat()

# Calling overridden methods
a.make_sound()   # Animal method
d.make_sound()   # Dog method
c.make_sound()   # Cat method
```

```
Animal makes a sound
Dog barks
Cat meows
```